

☒ Manual de Operaciones - Granja RPA en Linux (Versión CI/CD)

Esta guía contiene todos los comandos esenciales y scripts maestros (píldoras) para administrar, actualizar y monitorear tu ecosistema de bots RPA en tus servidores Ubuntu/Proxmox.

1. ☒ Conexión Remota (Desde Windows)

Abre PowerShell o CMD en tu computadora y usa el siguiente comando para entrar al servidor por consola.

```
ssh ugenome@IP_DE_TU_SERVIDOR
```

(Ejemplo: `ssh ugenome@192.168.1.150`).

- **Contraseña:** UbuntuCyb2026\$ (Para salir y volver a Windows, escribe `exit`).

[!WARNING] **¿No te da acceso o se queda cargando (Timeout)?** Si el comando se queda pensando infinitamente o te arroja un error de conexión rechazada, significa que la máquina virtual está **apagada en Proxmox** o completamente inaccesible por red. Tendrás que encenderla primero desde el panel de Proxmox.

2. ☒ Comandos de Monitoreo Diario

Usa estos comandos en la terminal de tu Linux para vigilar el comportamiento del ecosistema.

[!TIP] Gestión del Servicio General

Comprobar la salud del orquestador:

```
sudo systemctl status rpa-orchestrator
```

Reiniciar todo el ecosistema (Equivalente a apagar/prender):

```
sudo systemctl restart rpa-orchestrator
```

[!NOTE] Lectura de Bitácoras (Logs) en vivo (Usa `Ctrl + C` para dejar de verlos en vivo)

Ver el registro de Auto-Actualizaciones de Git:

```
tail -f /home/ugenome/RPA_linux/cambio.log
```

Ver las peticiones de Postman recibidas:

```
tail -f /home/ugenome/RPA_linux/flask.log
```

Ver el comportamiento del Orquestador (main3):

```
tail -f /home/ugenome/RPA_linux/main3.log
```

Ver mensajes generales (creación/reinicio del servicio por Systemd):

```
journalctl -u rpa-orchestrator -f
```

Ver los logs de un bot multi-slot (Ejemplo RPACNOS2):

```
# Puedes cambiar el número 1 por 2 o 3 para ver los otros procesos  
tail -f /home/ugenome/RPA_linux/RPACNOS2/rpa_bot_1.log
```

Ver los logs de un bot de un solo proceso (Ej. Rpa_suspensionTemp o Rpa_Not_Done_New):

```
# Log del guardián (tele.py)  
tail -f /home/ugenome/RPA_linux/NOMBRE_DEL_BOT/tele.log  
  
# Log del robot principal (main.py)  
tail -f /home/ugenome/RPA_linux/NOMBRE_DEL_BOT/watchdog.log
```

[!IMPORTANT] Supervisión de Procesos y RAM

Listar todos los scripts de python corriendo actualmente:

```
ps aux | grep python
```

*(¿Cómo saber si el sistema está sano? Al correr este comando deberías ver obligatoriamente la **Trinidad Base**: `main3.py`, `flask_izzi.py` y `watchdog.py`. Adicionalmente, por cada bot que esté en estado `STARTED`, deberías ver siempre una pareja: su respectivo `tele.py` y su `main.py`).*

Ejemplo de una salida sana (con un bot simple y un multi-slot activos):

```
ugenome ... python3 -u flask_izzi.py
ugenome ... python3 -u main3.py
ugenome ... python3 -u watchdog.py
ugenome ... python3 -u /home/ugenome/RPA_linux/Rpa_suspensionTemp/tele.py
ugenome ... python3 -u /home/ugenome/RPA_linux/Rpa_suspensionTemp/main.py
ugenome ... python3 -u /home/ugenome/RPA_linux/RPACNOS2/tele.py
ugenome ... python3 -u /home/ugenome/RPA_linux/RPACNOS2/main.py --slot 1
ugenome ... python3 -u /home/ugenome/RPA_linux/RPACNOS2/main.py --slot 2
ugenome ... python3 -u /home/
ugenome/RPA_linux/RPACNOS2/main.py --slot 3
```

Administrador de tareas visual (RAM y CPU en vivo):

```
htop
```

3. ☒ Comandos de Emergencia (Matar Procesos)

Si un bot se congela o necesitas forzar el reinicio de un script en particular sin reiniciar todo el ecosistema, usa la herramienta de francotirador `pkill -if`. El `-if` asegura que busque ignorando mayúsculas/minúsculas.

Matar TODAS las instancias de un bot específico:

```
pkill -if "RPACNOS2/main.py"
pkill -if "RPACNOS2/tele.py"
```

Matar a TODOS los bots del servidor (Botón de pánico):

```
# El orquestador y los watchdogs detectarán la caída y los revivirán
pkill -if "main.py"
pkill -if "tele.py"
pkill -if chrome
pkill -if chromedriver
```

4. 📦 Píldora Maestra de Auto-Actualización (Git CI/CD)

Este bloque se utiliza si deseas conectar un nuevo clon de Linux al repositorio de Git para que pueda auto-actualizarse con la orden **REINICIO**.

Paso 1: Configurar credenciales y preparar la carpeta

```
cd /home/ugenome/RPA_linux

# 1. Hacer que Git memorice el Token para siempre
git config --global credential.helper store

# 2. Crear .gitignore para no subir basura ni contraseñas
cat << 'EOF' > .gitignore
*.log
__pycache__/
*.pyc
.env
CuentasNO/
logs/
.vscode/
EOF

# 3. Inicializar Git y vincular la rama linux
git init
git checkout -b linux || git branch -M linux
git remote add origin https://github.com/AlbertoJAVL/Rpa_CX_Bots_Proxmox.git 2>/dev/null || git
remote set
-url origin https://github.com/AlbertoJAVL/Rpa_CX_Bots_Proxmox.git
```

Paso 2: Realizar el primer empuje (requiere ingresar el Token de Github)

```
git add .
git commit -m "Conectando servidor a rama Linux"
git push -u origin linux
```

Paso 3: Crear el script de Auto-Descarga (cambio.sh)

```

cat << 'EOF' > /home/ugenome/RPA_linux/cambio.sh
#!/bin/bash
sleep 3
echo "=== INICIANDO ACTUALIZACION DESDE GIT (RAMA LINUX) ==="
cd /home/ugenome/RPA_linux

git fetch origin linux
git reset --hard origin/linux
git pull origin linux

sed -i 's/^status=REINICIO/status=STARTED/g' /home/ugenome/RPA_linux/.env 2>/dev/null
sed -i 's/^status=REINICIO/status=STARTED/g'
"/home/ugenome/RPA_linux/C:\Rpa_CX_Bots_Proxmox\env" 2>/dev/null

echo "=== ACTUALIZACION TERMINADA, REINICIANDO ECOSISTEMA ==="

kill -f "python3 -u main3.py"
kill -f "python3 -u flask_izzi.py"
kill -f "python3 -u watchdog.py"
EOF
chmod +x /home/ugenome/RPA_linux/cambio.sh

```

5. ☒ Píldora de Guardianes Multi-Slot (3 Procesos a la vez)

Para los bots que requieren correr 3 instancias simultáneas sin corromper sus logs (RPACNOS2 y RPAFlagConfirmacion), se usa este guardián especializado:

```

# Ejemplo de inyección para RPACNOS2 (Aplica igual para RPAFlagConfirmacion cambiando los nombres)
cat << 'EOF' > /home/ugenome/RPA_linux/RPACNOS2/tele.py
# -*- coding: utf-8 -*-
import os, time, psutil, pytz, requests
from datetime import datetime

WATCHDOG_MEM_LIMIT = 400
TOKEN = "6819354375:AAFb2UuBWfbOkT83YDyt2IH_IHSUgOpnkuU"
CHAT = "-1002094293899"
TARGET = 3

def get_ip():
    try: return (__import__("subprocess").getoutput("hostname -I").split()+["127.0.0.1"])[0]
    except: return

```

```

"0.0.0.0"
ip = get_ip()

def send_msg(msg):
    try:
        t = datetime.now(pytz.timezone('America/Mexico_City')).strftime("%d-%m-%Y %H:%M:%S")
        requests.get(f
"https://api.telegram.org/bot{TOKEN}/sendMessage?chat_id={CHAT}&text=[RPACNOS2] {ip} | {msg}
| {t}", timeout=10)
    except: pass

def get_procs():
    slots = {}
    for proc in psutil.process_iter(['name', 'cmdline', 'memory_info']):
        try:
            if 'python3' in proc.info['name'].lower():
                cmd = " "
.join(proc.info.get('cmdline', [])).lower()
                if 'rpacnos2' in cmd and 'main.py' in cmd:
                    if '--slot 1' in cmd: slots[1] = proc
                    elif '--slot 2' in cmd: slots[2] = proc
                    elif '--slot 3' in cmd: slots[3] = proc
        except: pass
    return slots

def start_slot(s):
    os.system(f"nohup python3 -u /home/ugenome/RPA_linux/RPACNOS2/main.py --slot {s} >
/home/ugenome/RPA_linux/RPACNOS2/rpa_bot_{s}.log 2>&1 &"
)

def main():
    send_msg(f"Watchdog Multi-Instancia iniciado ({TARGET} bots)")
    while True:
        try:
            slots = get_procs()
            for s in range(1, TARGET + 1):
                if s not in slots:
                    send_msg(f"Slot {s} caído. Levantando...")
                    start_slot(s)
                    time.sleep(10)
                else:
                    ram = slots[s].memory_info().rss / (1024*1024)
                    if
ram > WATCHDOG_MEM_LIMIT:
                        send_msg(f"Slot {s} RAM {ram:.0f}MB excedida. Matando...")
                        try: slots[s].terminate(); slots[s].wait(5)
                        except: slots[s].kill()
                    time.sleep(10)
        except: time.sleep(10)

if __name__ == '__main__': main()
EOF

```

6. ☒ Píldora de Sistema Base (Systemd Inmortal)

Si instalas un clon desde cero, este es el comando para instalar Systemd con la configuración de "Restart=always".

```
sudo bash -c 'cat << EOF > /etc/systemd/system/rpa-orchestrator.service
[Unit]
Description=RPA Orchestrator y Flask API
After=network-online.target
Wants=network-online.target

[Service]
Type=forking
User=ugenome
WorkingDirectory=/home/ugenome/RPA_linux
ExecStart=/home/ugenome/RPA_linux/init.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF'

sudo systemctl daemon-reload
sudo systemctl enable rpa-orchestrator.service
sudo systemctl restart rpa-orchestrator.service
```